Individual Project

Design and Implementation of a Lightweight Kernel-Level Event Tracer and Profiler for Linux

Ondrej Hurinek

M00914542

Middlesex University

Faculty of Science and Technology

BSc Computer Science

Supervised by Michael Heeney

April 2026

Table of Contents

1. Introduction and Literature review
1.1 Introduction
1.2 Literature review
2. Requirements
2.1 Functional Requirements
2.2 Non-functional Requirements

3. Technical Background
3.1 Overview of Technical Background
3.2 System calls
3.3 Hardware interrupts
3.4 Software interrupts
3.5 Instruction pointer
3.6 Tools used in tracer
3.7 Linux tracepoints
3.8 TRACE_EVENT infrastructure
3.9 User space interfaces and Kernel libraries

4. Design
4.1 System call design
4.2 Interrupts design
4.3 Software interrupts design
4.4 Instruction pointer resolution design

5. Implementation
5.1 Common implementation patterns
5.2 System call tracer implementation
5.3 Interrupt tracer implementation
5.4 Software interrupt tracer implementation
5.5 Instruction pointer tracer implementation

6. Testing, evaluation and verification
6.1 System call tracer validation
6.2 Interrupt tracer validation
6.3 Software interrupt tracer validation
6.4 Instruction pointer resolution validation
6.5 Usability and design evaluation

7. Conclusion, educational contribution, limitations and future work
7.1 Conclusion
7.2 Educational contribution
7.3 Limitations
7.4 Future work

Appendix
Appendix A - Kernel modules lifecycles and tracepoint registrations
Appendix B - /proc usage in aggregation mode
Appendix C - Tracepoint definition and registration using TRACE_EVENT

Acknowledgements

I would like to thank Michael Heeney for his support throughout this project and my time at university.

From the first year, he encouraged my interest in low-level architecture and systems level programming, which had a big influence on the direction I took with this project. He also helped by recommendation and access to useful resources, including books that made it much easier to get started with topics like computer architecture, assembly level programming, and understanding how things work at a lower level.

His enthusiasm and approach to teaching helped me to stay motivated, especially during more difficult periods of my studies.

During this project, his feedback and guidance were very helpful, especially when dealing with difficult decisions through the implementation. His support made a big difference both, in completing this project and helped to reduce stress around its quality and deadlines.

I really appreciate the time and effort he has put into helping me develop these skills.

1. Introduction and Literature review

1.1 Introduction

Modern operating systems rely on complex interactions between user-space applications, the kernel, and hardware (Love, 2010). Understanding these interactions is important for debugging, analysis of software performance e.g. development of efficient software. However, many important system events, such as system calls, interrupts, and deferred kernel work, occur within kernel space and are not directly visible to the user for safety reasons (Bovet and Cesati, 2005). To allow user to visualise such events, kernel-level tracing tools are used to observe and analyse runtime behaviour of the operating system. These tools provide insight into how software interacts with system resources, how hardware events are handled, and where potential performance bottlenecks may occur. However, many existing tracing solutions either operate at a high level of abstraction or rely on complex frameworks, which can result in limitation of understanding of the underlying mechanisms.

The aim of this project is to design and implement a custom kernel-level tracing system using Linux tracepoints (The Linux Kernel Developers, 2024g). The developed tracer focuses on capturing and analysing system calls, hardware interrupts (IRQs) and software interrupts (softirqs). By combining these components, the tracer provides both high-level aggregated statistics and detailed event-based insights into system behaviour.

The system is implemented as a set of kernel modules. Each kernel module is responsible for a specific type of event. This modular approach allows independent development, testing, and possible extension of individual modules. The tracer supports two modes of operation: an aggregation mode for summarised analysis using the /proc filesystem (The Linux Kernel Developers, 2024b), and an event mode for detailed tracing using the kernel tracing subsystem (The Linux Kernel Developers, 2024c).

During development, issues regarding concurrent execution of system calls had to be considered with priority, to ensure correctness of the tracer, and maintain stability within kernel space. Multiple synchronization mechanisms and data structures were selected to handle high-frequency events safely and efficiently.

For verification of the correctness and effectiveness of the tracer, the collected data is compared and evaluated against already existing tracing tools such as strace, perf, and ftrace (Linux Manual Pages Project, 2025b; Linux Manual Pages Project, 2024; The Linux Kernel Developers, 2024d). This comparison is there to assure that the implemented modules produce accurate and correct results and provides additional validation of the tracing approach. Performance measurements are also considered to assess the runtime overhead produced by the tracer under extreme workload.

Although the primary focus of the project is on tracing system calls, interrupts, and software interrupts, the functionality was extended to include instruction pointer analysis. This feature goes slightly beyond traditional tracing software, but was included to provide deeper insight

into where system calls originate within user-space applications. By analysing instruction pointers, it has become possible to explore the actual execution point responsible for triggering them.

Overall, this project aims not only to provide a functional tracing tool, but also to explore and demonstrate how kernel-level observability can be implemented directly using Linux tracing mechanisms. Particular importance is given to knowledge gained during development, especially in system level programming and low level architecture. The resulting system offers valuable insight into low-level system behaviour and can be used for performance analysis, debugging, and other research into kernel instrumentation.

1.2 Related work / Literature review

Hausdorf et al. (2019) present SyCaT-Vis, a tool that helps analyse system call traces for security and anomaly detection. The paper discusses common detection approaches such as sequence matching, finite-state models, and machine learning techniques, but the main contribution is the visual exploration interface. SyCaT-Vis provides three linked views (context, process, and thread) that allow users to move from an overview to detailed syscall inspection, making it easier to spot unusual patterns in large datasets.

While this visual approach is powerful, this project takes a simpler direction. Instead of building a full graphical interface, the tracer focuses on producing clear, structured outputs that can be analysed through a command-line interface and optionally visualised externally. The goal is to keep the system lightweight, transparent, and focused on kernel-level tracing rather than UI complexity.

Gebai and Dagenais (2018) compare different Linux tracing tools and analyse their performance and design trade-offs. Their survey shows that static tracepoints and per-CPU ring buffers provide low overhead and good scalability.

This aligns with the choice to use static tracepoints / TRACE_EVENT rather than dynamic probes, to keep overhead low and behaviour stable across kernel versions.

Beamonte et al. (2012) evaluate the real-time performance of LTTng (Linux Trace Toolkit) 2.0 on multicore systems, focusing specifically on latency and determinism. Their results demonstrate that per-CPU buffers and minimal locking can maintain predictable overhead, but also show how certain synchronization mechanisms may introduce latency spikes. This reinforces the importance of measuring tracing overhead in this project’s evaluation phase.

Esteves et al. (2023) present DIO, a tool that observes storage-related system calls to diagnose application I/O problems like file-access mistakes and I/O contention. It traces syscalls with low intrusion, adds extra context (e.g., file offsets/types), and pushes events into a near real-time analysis/visualization pipeline.

For the project, this is useful as an example of how kernel-level tracing can be turned into actionable outputs (counts, timing, and "what's happening" patterns), and it reinforces the need for filtering and low-overhead collection when tracing high-frequency events.

2. Requirements

2.1 Functional requirements

- Capture system calls information, specifically duration of execution, frequency, total execution time, unresolved occurrences, and record the instruction pointer that invoked the system call
- Capture interrupt information, specifically duration, frequency, total execution time, and unresolved occurrences
- Capture software interrupts, measure their execution time, frequency, total execution time, and unresolved occurrences
- Trace and list hottest instruction pointers triggering system calls during execution

2.2 Non-functional requirements

- The tracer should maintain low overhead during the execution, especially while resolving and storing information about system calls, interrupts, software interrupts, and instruction pointers
- The system shall operate in kernel space exclusively, avoiding switching between kernel and user space for safety and performance reasons
- The tracer should correctly handle concurrency and multithreading when capturing information in kernel space
- The system shall produce clear, human-readable output without requiring additional tools from the user
- The tracer shall be developed as a set of individual modules, to assure that development and testing can be done separately

3. Technical Background

3.1 Overview of technical background

In this project, system calls, interrupts and software interrupts were observed, as well as low level CPU data were captured, such as view of the instruction pointer at the time of system call execution. These tools help identify issues in developed programs and can potentially be used in reverse engineering to map incorrect execution of the software, or in case of system

call, particular instruction. However, as with other kernel monitoring tools, this comes at the cost of increased runtime overhead.

3.2 System calls (syscalls) are main mechanism for user-space to interact with hardware (Love, 2010). If any software needs to use or change system resources, it does so by triggering system call, which serves as a transition from user-space into kernel (Bovet and Cesati, 2005). Although system calls introduce higher overhead due to introducing another layer of execution through runtime, they are especially important for the safety reasons, asuring system security by restricting user-space accessing kernel space directly.

3.3 Hardware interrupts (IRQs) are using interrupt signal to let the CPU know when their attention is needed (Mauerer, 2008). Examples include move of the mouse pointer, keyboard input, or packets arriving into the network card. When interrupt occurs, CPU temporarily stops lower priority execution and resolve triggered interrupt using interrupt handler inside the kernel space. These handlers are typically designed to do minimal work as they must be executed very quickly. Monitoring interrupts can help identify hardware issues or abnormal behaviour, such as devices generating excessive or unexpected interrupts.

3.4 Software interrupts (SOFTIRQs) - Since hardware interrupts must execute quickly and only run on single CPU, longer tasks are ‘deffered’ into software interrupts (Mauerer, 2008). Softirqs can run on multiple CPUs concurrently. They are also responsible for handling timers and deferred kernel work. In this project, I measure execution time of softirqs and track which CPU executes them.

3.5 The instruction pointer represents the exact location of the currently executing instruction within a program. From a tracing perspective, it provides a low-level view of where specific operations originate. In the context of this project, analysing instruction pointers allows identification of the actual execution point responsible for triggering system calls, rather than just the logical call site in user code. This makes it a valuable tool for understanding how programs interact with the operating system at a deeper level, and for exploring the relationship between user-space applications and kernel execution.

3.6 About the tools used in the tracer

Before implementing this tracer, several existing tracing tools were considered. However, they were not used for several reasons. First, some tools were too far out of scope for this project. Using them would require learning additional frameworks and mechanisms within very short period of time. Example of those were using kprobes, which allows probes to be attached to almost any kernel function at runtime (The Linux Kernel Developers, 2024e). Since they operate at a lower level than required, using those was decided as outside of scope for this project.

Second, some tools were too abstract, meaning that tools like ftrace, perf, eBPF, or bpftrace would provide powerful tracing tools, however, they operate at higher level and abstract many internal details (The Linux Kernel Developers, 2024d; The Linux Kernel Developers,

2024f). Therefore, they would limit the opportunity to learn about kernel internals, kernel modules, low level architecture and, in particular, used tracing mechanisms.

Third, some tools were not fully aligned with the goals of the project. Tools like strace and ptrace are effective, however they operate in user-space only, and they do not provide all of the information from requirements (Linux Manual Pages Project, 2025b; Linux Manual Pages Project, 2025a).

For these reasons, Linux tracepoints were chosen. They provide direct interaction with kernel tracing mechanisms, attaching probes directly to the tracepoints to access all of the details and information through runtime.

3.7 Linux Tracepoints are a tool implemented inside Linux kernel to provide mechanism to observe kernel events (The Linux Kernel Developers, 2024g). Each tracepoint represents a specific event. In this project, the most important tracepoints are sys_enter/sys_exit, irq_handler_entry, irq_handler_exit and softirq_entry, softirq_exit. Using those tracepoitns, probes were attached (registered) to these tracepoints to collect runtime data.

3.8 TRACE_EVENT Infrastructure extends the functionality of the tracepoints and allows them to automatically record the data inside kernel tracing subsystem (The Linux Kernel Developers, 2024c). When those events are triggered, the data are written into a per-CPU ring buffer (The Linux Kernel Developers, 2024g). Each CPU writes into its own buffer, so I do not need to worry about mixing events from multiple CPU's into 1 buffer.

3.9 User-Space Interfaces and Kernel libraries - To access collected data, /proc (The Linux Kernel Developers, 2024b) file system have been used for aggregated tracing, and tracefs (The Linux Kernel Developers, 2024g) to expose data stored in tracing ring buffers. /proc file system is commonly used to print tables of diagnostic data, while tracefs provides access to per-CPU ring buffer. To ensure that execution stays only in kernel space, several kernel libraries are used.

4. Design choices to reduce runtime overhead and ensure accuracy

Through development, few design choices were considered, mainly due to consideration of multithreading and concurrency, to assure correctness of the tracer.

4.1 System calls

When designing system call tracing modules, the goal was to count system call executions, but also measure execution time. To achieve that, timestamps were stored at system call enter and system call exit, but this approach have introduced important issue.

To correctly measure system call exit to already started entry point, awareness was raised to the fact that there can be more than one system call executing at the same time with the same id concurrently. Therefore, the only way to connect exit to the entry was via thread id, that has been used as key in hash table. I have used hash table for O1 complexity at the lookup

function. A kernel spinlock have been used when modifying the hash table, to make sure that hash table was not modified concurrently on different CPUs at once (Love, 2010). Spinlock is a way to block specific part of instruction in the code to be executing concurrently. The main reason for this is, that each hash table element also stores its node, that is part of the data chain, and if other CPU was dependent on that particular node, and it gets modified, the data would be incorrect.

To store system calls in aggregated mode, simple array have been used, with index corresponding to the system call id. The array is of a type struct, so each element stores all important data, that is than printed to the final table. To store counters and other information about the system calls, atomic counters have been used, because two or more system calls could be updating variables at the same time. This have assured accuracy of the final results.

In emission layer, all of the information about the system calls have been printed directly at the exit.

4.2 Interrupts

Similar to system calls, the aim was to track number of executed interrupts and measure the time of their execution. Unlike system calls, interrupts execute exclusively per CPU, therefore there was no need to resolve concurrency issues regarding connectivity of interrupt entry with interrupt exit.

However, issues were still there, since interrupts can still execute on the same CPU simultaneously, so atomic structures were necessary to use. The main challenge was with handling interrupts that occur through the execution of other interrupts. To resolve this issue, n-sized array with depth n have been used to store nested interrupts in stack-like manner. When an interrupt with higher priority occurs, the time of the executing interrupt is stored, and restored once execution of the higher priority interrupt is done. Arranging interrupts this way, there was no need to worry about interrupts being interrupted with those of the lower priority, since they will not even start execution, and will wait in a queue until they are ready to be executed.

Same as in system call case, in aggregated mode, I have stored interrupts in an three dimensional array. First dimension represent CPU, so I can store individual interrupts per CPU. Second represent interrupt number, since each interrupt has its own id. Third is the interrupt itself, since same id can be shared between multiple devices. These elements are check by comparing their device id field.

In emission layer, same as in system call context, I have printed all information directly at the interrupt exit.

4.3 Software interrupts

To store information about software interrupts, advantage has been taken of the fact that, just like interrupts, there can not be one software interrupt executed on the same CPU at the same

time. The only issue that had to be considered was, that there can be a software interrupt with the same name executed parallelly on the different CPUs. Therefore, to avoid incorrectness (meaning two CPUs updating the same software interrupt at the same time), they have been stored in two dimensional array, with first dimension being the CPU, and the second the software interrupt vector number.

Printing software interrupts is done the same way as in interrupts and system calls context.

4.4 IP analysis

The instruction pointer analysis extends the previous system call tracing module by showing, which instruction exactly did the system call fired at the user-space. To achieve this, the module reads instruction pointer register at the system call entry, which represents the exact memory address of the code snipped that called the system call. Hash table have been used to store information about the instruction pointer, with the combination of instruction pointer, process id and system call id as a key. Access and modification of hash table is protected by spinlock, to assure that data modified are correct, since the system calls may be invoked simultaneously on different processors. As before, I have implemented the module such way that it uses the spinlock for the shortest possible time possible to assure that the CPU does not stay locked for too long. Such incorrect usage of spinlock could introduce high overhead.

However, accessing the instruction pointer alone is not sufficient to track down the instruction when reverse engineering the executable. System calls might be triggered by other modules or libraries called by executable, and the information stored would not be able to see that. For that reason, only, image resolution function was implemented.

Image resolution is responsible to get information about the exact module or library that is responsible for calling the system call using the instruction pointer listed by the tracer. To achieve this information, I have compared my traced instruction pointer to the corresponding virtual memory area and extracted associated file name (Bovet and Cesati, 2005). Since this operation requires memory mapping lock (similar to the spin lock, but to assure that memory map stays the same and does not change until my mapping is done) and looping through process address space structures (to compare my instruction pointer with process address space structures as mentioned before), it have introduced incredibly high overhead. For that very reason, the decision have been made to only allow image resolution if user have specified tracing of specific system call and process id.

To resolve the final correct instruction pointer, the base address of the corresponding memory mapping was subtracted from the captured instruction pointer, producing final instruction pointer offset.

For aggregation level output, the hash table is converted into a temporary array, sorted by entries count and displayed top N most frequent calls using /proc interface. For emission layer, instruction pointer with the file name together with the system call was printed.

5. Implementation

The modules were built using the standard Linux kernel build system. Each module includes a Makefile that compiles the source code against the currently running headers. Modules are inserted using insmod and removed using rmmod. This allows loading the module through runtime, so rebooting system is not required.

5.1 Common implementation patterns

Each kernel module has been built to trace different data, however, some implementation techniques were shared:

Tracepoint lookup by name - Instead of referring to the tracepoints directly, all modules dynamically locate tracepoints at runtime using their names. This was achieved by iterating through all defined tracepoints inside the kernel, comparing their names. With this approach, it is much easier to adapt modules to different kernel configurations.

Probe registration and unregistration - Once a tracepoint is found, a probe function is registered using ‘tracepoint_probe_register’ function. This function is called whenever the tracepoint fires, meaning, on each system call entry, ‘syscall_enter’ function runs.

During module unload, all probes are properly unregistered using ‘tracepoint_probe_unregister’ function to make sure no callback functions are still executing.

Data exposure using /proc and trace events - to expose data, two approaches were used:

- Aggregation mode - Results were stored internally using local (global in terms of module scope) variables, exposed using /proc filesystem into easy-to-read tables

- Event mode - Events are stored inside per-CPU ring buffers using TRACE_EVENT infrastructure, which are than accessed using tracefs filesystem.

This separation allows for lightweight monitoring and detailed tracing.

Locking and synchronization - Different synchronization strategies are used, depending on the module and what they trace:

- Spinlocks are used to protect shared data structures such as hash tables

- Atomic operations counters and numeric values updated concurrently

- Per-CPU data structures are used where possible to avoid locking and using atomic operations, since they are lighter for execution

Safety considerations - Several mechanisms were used to protect data and incorrect memory access:

- Validations of assigned data
- Error handling
- Bounds checking
- Proper cleanups of allocated memory
- Use of kernel-safe allocation flags (e.g. GFP_ATOMIC) wherever required

5.2 System call tracer implementation

The system call tracer is implemented using two tracepoints: sys_enter and sys_exit. These tracepoints trigger placed callback functions, that collects, compare and expose all important data.

System call entry handling

When a system call is invoked, the sys_enter tracepoint triggers the probe functions. At this point, the module executes several steps in order to collect data. First, if a specific process (target_pid) or system call (target_syscall) is entered (by the user at the module load up), conditions are checked, deciding what data, if any are to be collected. This allows selective tracing and reduces overhead. Next, the current timestamp is stored in the local variable using ktime_get_ns(), and the instruction pointer is stored in the variable using instruction_pointer(regs). The module also records process name and, if enabled, resolve the instruction pointer to a user-space file image. A new struct variable is than allocated to represent particular instance. The structure contains:

- Thread ID
- Syscall ID
- Start timestamp
- Instruction pointer
- Process name
- Image name

The module than uses a spinlock and checks whether the struct already exists in hash table. If it does, this means that a previous system call was not properly matched with an exit event. In this case, the unresolved counter is incremented and the existing struct is than overwritten with the new data. If no entry exists, the new struct is added into the hash table using the thread ID as the key. The spin lock is than released.

System call exit handling

When the system call completes its execution, the sys_exit tracepoint triggers the exit probe. The module than applies filtering, same as at the entry, and retrieves the current timestamp. It than calls spinlock and looks up the corresponding entry in the hash table using thread ID. If not matching is found, the event is ignored. Otherwise, the duration of the system call is calculated by subtracting the stored start timestamp from the current time. The entry is than removed from the hash table and the spinlock is than released. After that, the module updates aggregated statistics:

- Increment the system call count
- Adds the duration to the total execution time

After all, temporary structure is freed.

Aggregate mode data

Aggregate data are stored in an array, which index represents system call ID. Each element of the array contains:

- Total count
- Total execution time
- Unresolved occurrences

These values are updated using atomic operations. The aggregated results are than exposed using /proc filesystem, where they are formatted into a readable table including system call name, ABI, counts, unresolved cases, average time, and total time.

Event mode data

When a system call completes its execution, the tracepoint callback calls the function trace_sys_tracer_only to write following data into the ring buffer:

- Syscall ID and name
- Execution duration
- Invocation count
- Resolved/unresolved state
- Process ID and name
- Instruction pointer

- Resolved image name

5.3 Interrupt tracer implementation

The IRQ tracer is implemented using irq_handler_entry and irq_handler_exit traceoints to capture the beginning and the end of interrupt handling. This allows the module to capture desired data through the execution.

Interrupt entry handling

When an interrupt occurs, the irq_handler_entry tracepoint triggers the entry probe. Basic validations are than performed to ensure that the interrupt number does not exceed the predefined bounds and that the per-CPU stack has not exceeded its maximum depth (since they are both hardcoded, as they are used as array indexes, and must have number on the array definition). A temporary structure is than created to represent the current interrupt. This structure contains:

- Interrupt ID
- Device name
- Start timestamp
- Device identifier

If another interrupt was already executing on the same CPU (i.e. the stack-like array is not empty), the module updates the execution time of the current (previous in the new interrupt context) interrupt by adding the elapsed time up to the moment it was interrupted. This assures correct execution time even during nested interrupts execution. The new interrupt is than pushed into the per-CPU stack-like array by incrementing the array pointer and storing the structure at the top of the stack.

Interrupt exit handling

When the interrupt completes execution, the irq_handler_exit tracepoint triggers the exit probe. The module than validates top of the stack-like array state to ensure that there is a matching interrupt entry. The stack-like array is than updated to remove the interrupt on the top. The module than updates the results stored in 3D array. The array is indexed by:

- CPU
- Interrupt number
- Device ID

To identify the correct interrupt inside the array, the module uses CPU and interrupt number index, and compares device ID. If struct exists, the counters are updated. If it does not, the

new instance is created inside the array. If the completed interrupt had interrupted another interrupt, the module restores execution of the previous interrupt by updating its start timestamp to the current time. At the end, the stack pointer is decremented, since the interrupt has completed.

Aggregate mode data

Since all important data are already stored in the previously mentioned three-dimensional array, there is no need to create another data structure for aggregate mode. This structure allows to separate interrupts with the same ID, but different devices. Each element inside the array contains:

- Number of observed executions
- Total execution time
- Device name

The aggregate results are exposed using /proc filesystem. The output includes CPU number, interrupt ID, number of executions, average execution time, total execution time and device name.

Event mode data

When an interrupt completes its execution, the tracepoint callback calls the function trace_irq_tracer_only to write following data into the ring buffer:

- Interrupt ID
- Device name
- Execution time
- Total accumulated time
- Number of occurrences

5.4 Softirq tracer implementation

The software interrupt tracer is implemented using the softirq_entry and softirq_exit tracepoints. These tracepoints trigger callback functions once softirqs begin and ends to collect all necessary data.

Software interrupt entry handling

The module first checks if the softirqs vector number is within the supported range. If the vector number is not valid, the event is ignored. The tracer then checks the current_softirq array for the current CPU and vector. This array stores the start timestamp of the currently

active softirq for that CPU. If that slot is not 0, it means that the softirq is already stored there, and was not correctly resolved before a new one started its execution. In this case, the unresolved counter is incremented, and the start time of the current_softirq array is set to the current time, marking the start of the new softirq execution.

In event mode, if unresolved case happens, the module prints data of the unresolved softirq immediately.

Software interrupt exit handling

When the softirq finishes execution, the softirq_exit tracepoint triggers the exit probe. The module first validates the vector number. It than checks whether a matching start timestamp exists in current_softirq for the current CPU vector. If the stored value is zero, this means that the tracer observed a software interrupt exit without a corresponding recorded entry. In this case, the unresolved counter is incremented and the event is ignored. If a valid start timestamp exists, the module calculates the execution time by subtracting the stored start time from the current time. It than updates result for that CPU and vector:

- Stores the softirq name
- Increments the observed count
- Adds duration to the total execution time

At the end, the current_softirq slot is reset to zero (which indicates that there is no active softirq at the moment).

Aggregate mode data

Aggregated statics are printed from the previously mentioned 2D array, indexed by CPU and softirq vector. The aggregate results are printed through /proc filesystem. The output includes CPU number, softirq name, observed count, unresolved count, average time, and total execution time.

Event mode data

In event emission mode, each event contains:

- CPU number
- Softirq vector number
- Softirq name
- Execution time
- Observed count

- Unresolved count

Those are written into per-CPU ring buffer, and can be observed using tracefs subsystem.

5.5 Instruction pointer tracer implementation

The instruction pointer tracer extends the system call tracer by resolving the origin of the system calls. Instead of measuring the execution time, this module tracks down how often specific instruction pointers call system calls. It has been implemented using sys_enter tracepoint, since there is no need to compare enter with exit.

System call entry handling

When a system call is triggered, the sys_enter tracepoint activates the probe. The module starts with validating syscall ID and applies filters for target_pid and target_syscall. The instruction pointer is then extracted from the register using instruction_pointer(regs). The module records:

- Process ID
- Syscall ID
- Process name

A hash table key is created combining instruction pointer, system call ID and process ID. The module than uses spinlock and uses the key for the existing entry lookup inside the hash table. If the entry exists, its counter is incremented and the probe returns immediately. If it does not, the module releases the lock and alocates the new structure to represent this call.

Instruction pointer resolution (integration with syscall tracer)

Once this new structure has been allocated, the module optionally (depending on the user setting for process ID and syscall ID) resolves the instruction pointer to a user-space image. This was intended to prevent the system to crash, since the image resolution introduce extremely high overhead while executing system calls. The resolution is performed by looping through the process memory mappings:

- The process memory descriptor (mm_struct) is accessed
- A read lock is acquired using mmap_read_lock
- The correct virtual memory area (VMA) is located using find_vma
- If a valid mapping is found, the associated file name is extracted from the VMA

This mechanism is similar to the system call tracer, where instruction pointer resolution is also conditionally enabled to reduce overhead. However, in this module, resolution is performed only once per unique call rather than per system call execution.

Hash table update

After preparing the new entry, the module uses spinlock again for another hash table lookup. If the searched struct exists, counter is incremented. Otherwise, the new struct is inserted into the hash table.

Aggregate mode data

The module reads results directly from the hash table. When the /proc filesystem is called, the module converts hash table into a temporary array for sorting. It then prints an array:

- Rank
- Number of occurrences
- Process ID
- System call ID
- Instruction pointer
- Process name
- Resolved image name

Helper module - System call Tables

A helper module was implemented for easier access about the information about the system calls, specifically their names and ABI. The module exposes two helper functions:

- get_syscall_name(int id) - returns the name of the system call
- get_syscall_abi(int id) - returns the ABI type of a system call

These functions are exported using EXPORT_SYMBOL, otherwise, compiler would not be able to see them, since they are loaded as all other kernel modules, and are not seen unless the kernel module runs.

6. Testing, evaluation and verification

For testing and evaluation, the focus was on correctness of the tracer, its runtime overhead and simplicity of use. All tests were performed on a Virtual Machine system using Oracle Virtual Box, running Ubuntu version 22.04.5 LTS (long term support) for stability, with a kernel version 6.8.0-106-generic. The modules were compiled using GCC (version 11.4.0),

while the frontend (CLI interaction) was compiled using the same compiler for C++. The virtual machine was configured with 8 GB of memory and 8 CPU cores.

Each module (system call tracer, IRQ tracer, softirq tracer and instruction pointer tracer) was tested independently to ensure correctness of functionality. Both, aggregation mode and event mode, were evaluated separately in order to compare their behaviour and impact on system performance.

As mentioned earlier, the tracing modules were tested against the following tools:

- Strace - used to trace system calls at the user-space level, providing a reference for validating syscall counts and vehaviour
- Perf - used for performance analysis and event counting, helping to verify system activity
- Ftrace - a kernel-level tracing framework used to observe internal kernel events and validate low-level behaviour
- Tracepipe (raw tracepoints) - used to directly observer tracepoint output and confirm correctness of captured events

Testing also focused on measuring runtime overhead and identifying potential limitations of the implementation, that were accepted or not considered during development. The overall goal of these evaluation methods was to ensure that the tracer produces accurate results under concurrent and multithreaded workloads, while maintaining relatively acceptable performance.

6.1 System call tracer evaluation

The correctness of the system call tracer was validated by comparing its output with the Strace tool. Two small pieces of code were written to generate a predictable set of system calls. A code included running a combination of file operations and repeated I/O calls, ensuring that multiple system call types were called during execution. Both tracers, Strace and custom developed tracer, were attached to the running software using its PID. I was able to attach both tracers at the exact time during the execution. The number of executed system calls were than compared from both tools:

```
ondrej     22169  0.0  0.0   2776  1296 pts/0    S+   21:13   0:00 ./a.out
ondrej     22171  0.0  0.0  12688  3632 pts/1    R+   21:14   0:00 ps aux
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ ./tracer.out -m sys_summary -p 22169
Loading module: sudo insmod aggregated_mode/sys_summary/syscall_summary_mod.ko target_pid=22169
Tracing... (p = print /proc (agg mode only), q = quit)

===== /proc snapshot =====
-----------------------------------------------------------------------------------------------------------
  ID |             SYSCALL NAME | SYSCALL ABI |  OBS TIMES |      UNRESOLVED |        AVG TIME |      TOTAL TIME
-----------------------------------------------------------------------------------------------------------
   0 |                     read |      common |          1 |               0 |      4220084655 |      4220084655
   1 |                    write |      common |       1000 |               0 |            1737 |         1737693
   3 |                    close |      common |          1 |               0 |          127225 |          127225
 257 |                   openat |      common |          1 |               0 |           16543 |           16543
==========================

Stopping tracer...
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ ./tracer.out -m sys_summary -p 22236
Loading module: sudo insmod aggregated_mode/sys_summary/syscall_summary_mod.ko target_pid=22236
Tracing... (p = print /proc (agg mode only), q = quit)

===== /proc snapshot =====
-----------------------------------------------------------------------------------------------------------
  ID |             SYSCALL NAME | SYSCALL ABI |  OBS TIMES |      UNRESOLVED |        AVG TIME |      TOTAL TIME
-----------------------------------------------------------------------------------------------------------
   0 |                     read |      common |          2 |               0 |      1965998855 |      3931997710
   1 |                    write |      common |          2 |               0 |           25244 |           50489
   3 |                    close |      common |          1 |               0 |           34224 |           34224
  39 |                   getpid |      common |          1 |               0 |             657 |             657
  74 |                    fsync |      common |          1 |               0 |         2403761 |         2403761
 110 |                  getppid |      common |          1 |               0 |            1976 |            1976
 230 |          clock_nanosleep |      common |          1 |               0 |         2833645 |         2833645
 257 |                   openat |      common |          1 |               0 |           83444 |           83444
 262 |               newfstatat |      common |          1 |               0 |            3783 |            3783
==========================
```

```
ondrej@ondrej-VirtualBox:~$ sudo strace -c -p 22169
[sudo] password for ondrej:
strace: Process 22169 attached
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 93.17    0.002566           2      1000           write
  4.68    0.000129         129         1           close
  1.49    0.000041          41         1           read
  0.65    0.000018          18         1           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.002754           2      1003           total
ondrej@ondrej-VirtualBox:~$ sudo strace -c -p 22236
strace: Process 22236 attached
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         2           read
  0.00    0.000000           0         2           write
  0.00    0.000000           0         1           close
  0.00    0.000000           0         1           getpid
  0.00    0.000000           0         1           fsync
  0.00    0.000000           0         1           getppid
  0.00    0.000000           0         1           clock_nanosleep
  0.00    0.000000           0         1           openat
  0.00    0.000000           0         1           newfstatat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        11           total
ondrej@ondrej-VirtualBox:~$
```

The results showed that the system calls produced by both, custom tracer and Strace tool, were counted the same way through the execution of both programs. This confirms that the system call tracer correctly captures and aggregates system call activity without significant loss of events. To compare the emission layer results, tracepipe mechanism was used to compare inbuild Linux trace mechanisms with my tracer:

```
root@ondrej-VirtualBox:~# cat /sys/kernel/tracing/trace_pipe
          a.out-22615   [006] ...1. 10590.840069: sys_read -> 0x1
          a.out-22615   [006] ...1. 10590.840161: sys_openat(dfd: ffffff9c, filename: 641bf4fce025, flags: 241, mode: 1a4)
          a.out-22615   [006] ...1. 10590.840250: sys_openat -> 0x3
          a.out-22615   [006] ...1. 10590.840333: sys_write(fd: 3, buf: 641bf4fce02e, count: 6)
          a.out-22615   [006] ...1. 10590.840349: sys_write -> 0x6
          a.out-22615   [006] ...1. 10590.840350: sys_fsync(fd: 3)
          a.out-22615   [006] ...1. 10590.845543: sys_fsync -> 0x0
          a.out-22615   [006] ...1. 10590.845634: sys_close(fd: 3)
          a.out-22615   [006] ...1. 10590.845642: sys_close -> 0x0
          a.out-22615   [006] ...1. 10590.845646: sys_getpid()
          a.out-22615   [006] ...1. 10590.845646: sys_getpid -> 0x5857
          a.out-22615   [006] ...1. 10590.845647: sys_getppid()
          a.out-22615   [006] ...1. 10590.845648: sys_getppid -> 0xec0
          a.out-22615   [006] ...1. 10590.845649: sys_newfstatat(dfd: ffffff9c, filename: 641bf4fce025, statbuf: 7ffe64aadb50, flag: 0)
          a.out-22615   [006] ...1. 10590.845653: sys_newfstatat -> 0x0
          a.out-22615   [006] ...1. 10590.845654: sys_clock_nanosleep(which_clock: 0, flags: 0, rqtp: 7ffe64aadb10, rmtp: 0)
          a.out-22615   [006] ...1. 10590.848120: sys_clock_nanosleep -> 0x0
          a.out-22615   [006] ...1. 10590.848217: sys_write(fd: 1, buf: 641c1c1962a0, count: 1d)
          a.out-22615   [006] ...1. 10590.848224: sys_write -> 0x1d
          a.out-22615   [006] ...1. 10590.848225: sys_read(fd: 0, buf: 641c1c1966b0, count: 400)
          a.out-22615   [006] ...1. 10595.999166: sys_read -> 0x1
          a.out-22615   [006] ...1. 10595.999246: sys_exit_group(error_code: 0)
```

```
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ sudo ./tracer.out -m syscall_event -p 23214
insmod: ERROR: could not insert module common/trace_tables/trace_tables.ko: File exists
Loading module: sudo insmod event_mode/sys_summary_event/sys_summary_event_mod.ko target_pid=23214
Tracing... (p = print /proc (agg mode only), q = quit)
          <...>-23214   [004] ...1. 11350.662543: sys_tracer_only: id=257 name=openat duration=128883 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out
proc ip=0x7b8416d145bb image name=undefined
          <...>-23214   [004] ...1. 11350.662721: sys_tracer_only: id=1 name=write duration=23348 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out proc
ip=0x7b8416d14907 image name=undefined
          a.out-23214   [004] ...1. 11350.677443: sys_tracer_only: id=74 name=fsync duration=14718265 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out
proc ip=0x7b8416d1b907 image name=undefined
          a.out-23214   [004] ...1. 11350.677577: sys_tracer_only: id=3 name=close duration=35457 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out proc
ip=0x7b8416d14fe7 image name=undefined
          a.out-23214   [004] ...1. 11350.677580: sys_tracer_only: id=39 name=getpid duration=518 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out proc
ip=0x7b8416cec04b image name=undefined
          a.out-23214   [004] ...1. 11350.677582: sys_tracer_only: id=110 name=getppid duration=1305 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out p
roc ip=0x7b8416cec05b image name=undefined
          a.out-23214   [004] ...1. 11350.677589: sys_tracer_only: id=262 name=newfstatat duration=6008 count=1 resolved=true unresolved=0 process id=23214 proc name=a.ou
t proc ip=0x7b8416d13dbe image name=undefined
          a.out-23214   [004] ...1. 11350.679630: sys_tracer_only: id=230 name=clock_nanosleep duration=2038964 count=1 resolved=true unresolved=0 process id=23214 proc n
ame=a.out proc ip=0x7b8416ce578a image name=undefined
          a.out-23214   [004] ...1. 11350.679738: sys_tracer_only: id=1 name=write duration=9112 count=2 resolved=true unresolved=0 process id=23214 proc name=a.out proc
ip=0x7b8416d14907 image name=undefined
          a.out-23214   [004] ...1. 11352.689228: sys_tracer_only: id=0 name=read duration=2009522104 count=1 resolved=true unresolved=0 process id=23214 proc name=a.out
proc ip=0x7b8416d14862 image name=undefined
```

It is important to note, that the frontend does not automatically reset the global tracefs state before starting event-based tracing. While clearing the trace buffer and disabling previously enabled events can make testing easier, it affects the tracing subsystem globally and may interfere with other tracing sessions already running on the system. For this reason, tracefs cleanup was done manually during testing, where no other tracing activity was expected, and this procedure is recommended (but not required) before each tracing for accurate data.

The event-based system call tracer was validated by comparing its output with the raw tracepoints available through trace_pipe. The raw output shows all system call entries and exits recorded by the kernel, including arguments and return values. In comparison, the custom tracer produces structured output, including execution time, process information, and resolved instruction pointer data. From the comparison, both outputs show the same sequence and types of system calls (such as read, write, openat, close, fsync). The number of system calls and their order also match, which confirms that the tracer correctly hooks into the sys_enter and sys_exit tracepoints.

The main difference is in how the data is presented. The trace_pipe includes all available raw data defined in the tracepoint, and split system call enter and system call exit. Custom tracer takes that information and prints it in more readable way. It also measures the time of the execution, which I believe makes the output more suitable for analysis, while still preserving capabilities of the raw kernel tracers. Overall, both, the aggregated and event-based tracers show that the system call tracers capture correct system calls, and work correctly.

6.2 Interrupt tracer evaluation

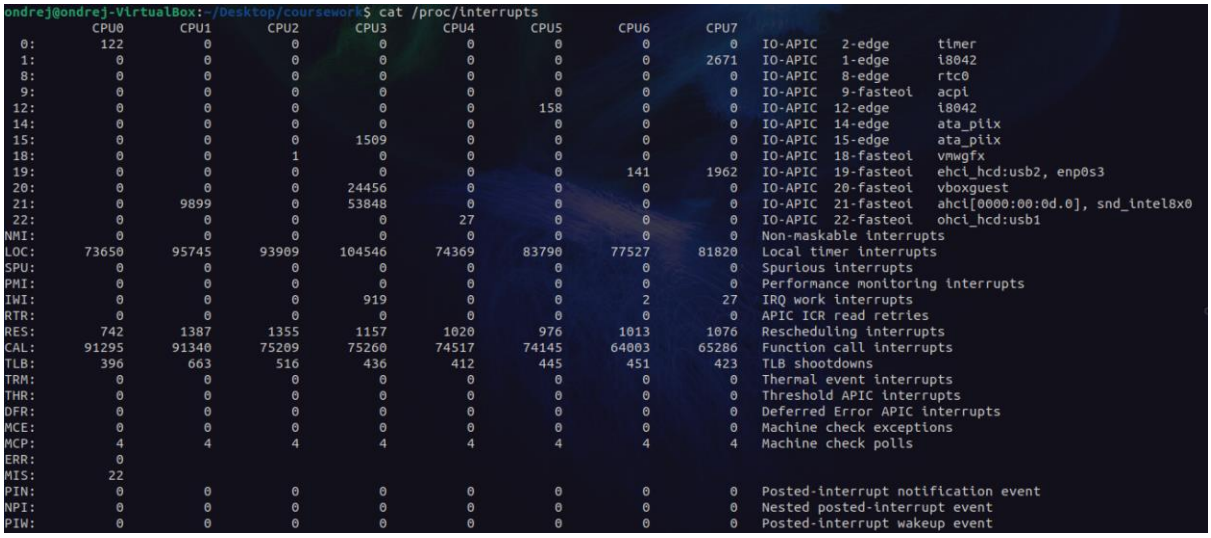
The interrupt tracer was validated by comparing both aggregation mode and event mode outputs with multiple kernel-level sources. Since interrupts cannot be fully verified using a single tool, different tools were used to validate specific aspects of correctness. Interrupt counts were compared against /proc/interrupts, which provides per-CPU statistics directly from the kernel. The number of observed interrupts and how they were distributed across CPUs in the tracer matched the values reported by /proc/interrupts, confirming that interrupts were correctly captured and counted.

In event mode, the tracer output was compared with raw interrupt tracepoints observed through trace_pipe. By enabling IRQ-related events in trace file system, it was possible to see interrupt activity in real time. The tracer produced events in the same order and occurrence patterns as those observed in trace_pipe, showing that it correctly hooks into irq_handler_entry and irq_handler_exit.

Unlike system calls, interrupts execute in interrupt context and are handled per CPU. This means that only one interrupt handler executes at a time on a given CPU. This simplifies matching entry and exit events. However, nested interrupts can still happen when a higher-priority interrupt preempts a currently executing one. The tracer handles this by using a per-CPU stack-like structure, which allows it to correctly measure execution time even in these cases.

To analyse execution behaviour, the tracer results were compared with kernel tracing output using ftrace and inbuilt /proc/interrupts file. By enabling interrupt tracing events, it was possible to observe approximate interrupt execution durations and timing behaviour. Changes in /proc/interrupts were compared with the aggregated tracer output while generating interrupts using a ping command. Some background interrupts (such as timers or keyboard input) were always present, but the main patterns still matched.

These are the interrupts shown before triggering some additional interrupts using ping:



```
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
  0:        122          0          0          0          0          0          0          0   IO-APIC   2-edge      timer
  1:          0          0          0          0          0          0          0       2671   IO-APIC   1-edge      i8042
  8:          0          0          0          0          0          0          0          0   IO-APIC   8-edge      rtc0
  9:          0          0          0          0          0          0          0          0   IO-APIC   9-fasteoi   acpi
 12:          0          0          0          0          0        158          0          0   IO-APIC  12-edge      i8042
 14:          0          0          0          0          0          0          0          0   IO-APIC  14-edge      ata_piix
 15:          0          0          0       1509          0          0          0          0   IO-APIC  15-edge      ata_piix
 18:          0          0          1          0          0          0          0          0   IO-APIC  18-fasteoi   vmwgfx
 19:          0          0          0          0          0          0        141       1962   IO-APIC  19-fasteoi   ehci_hcd:usb2, enp0s3
 20:          0          0          0      24456          0          0          0          0   IO-APIC  20-fasteoi   vboxguest
 21:          0       9899          0      53848          0          0          0          0   IO-APIC  21-fasteoi   ahci[0000:00:0d.0], snd_intel8x0
 22:          0          0          0          0         27          0          0          0   IO-APIC  22-fasteoi   ohci_hcd:usb1
NMI:          0          0          0          0          0          0          0          0   Non-maskable interrupts
LOC:      73650      95745      93909     104546      74369      83790      77527      81820   Local timer interrupts
SPU:          0          0          0          0          0          0          0          0   Spurious interrupts
PMI:          0          0          0          0          0          0          0          0   Performance monitoring interrupts
IWI:          0          0          0        919          0          0          2         27   IRQ work interrupts
RTR:          0          0          0          0          0          0          0          0   APIC ICR read retries
RES:        742       1387       1355       1157       1020        976       1013       1076   Rescheduling interrupts
CAL:      91295      91340      75209      75260      74517      74145      64003      65286   Function call interrupts
TLB:        396        663        516        436        412        445        451        423   TLB shootdowns
TRM:          0          0          0          0          0          0          0          0   Thermal event interrupts
THR:          0          0          0          0          0          0          0          0   Threshold APIC interrupts
DFR:          0          0          0          0          0          0          0          0   Deferred Error APIC interrupts
MCE:          0          0          0          0          0          0          0          0   Machine check exceptions
MCP:          4          4          4          4          4          4          4          4   Machine check polls
ERR:          0
MIS:         22
PIN:          0          0          0          0          0          0          0          0   Posted-interrupt notification event
NPI:          0          0          0          0          0          0          0          0   Nested posted-interrupt event
PIW:          0          0          0          0          0          0          0          0   Posted-interrupt wakeup event
```

```
===== /proc snapshot =====

 CPU |      IRQ NUMBER |   OBSERVED TIMES |     AVERAGE TIME |      TOTAL TIME |                 DEVICE NAME
-------------------------------------------------------------------------------------------------------------
   3 |              15 |                2 |           206819 |          413638 |                    ata_piix
   3 |              20 |                2 |            25173 |           50346 |                   vboxguest
   7 |               1 |                5 |            68002 |          340010 |                        i8042
   7 |              19 |                1 |            38291 |           38291 |              ehci_hcd:usb2
   7 |              19 |                1 |            50137 |           50137 |                    enp0s3
==========================
```

And these are the same interrupts after using ping:

```
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
  0:        122          0          0          0          0          0          0          0   IO-APIC   2-edge      timer
  1:          0          0          0          0          0          0          0       2797   IO-APIC   1-edge      i8042
  8:          0          0          0          0          0          0          0          0   IO-APIC   8-edge      rtc0
  9:          0          0          0          0          0          0          0          0   IO-APIC   9-fasteoi   acpi
 12:          0          0          0          0          0        158          0          0   IO-APIC  12-edge      i8042
 14:          0          0          0          0          0          0          0          0   IO-APIC  14-edge      ata_piix
 15:          0          0          0       1545          0          0          0          0   IO-APIC  15-edge      ata_piix
 18:          0          0          1          0          0          0          0          0   IO-APIC  18-fasteoi   vmwgfx
 19:          0          0          0          0          0          0        141       2022   IO-APIC  19-fasteoi   ehci_hcd:usb2, enp0s3
 20:          0          0          0      24524          0          0          0          0   IO-APIC  20-fasteoi   vboxguest
 21:          0       9899          0      53913          0          0          0          0   IO-APIC  21-fasteoi   ahci[0000:00:0d.0], snd_intel8x0
 22:          0          0          0          0         27          0          0          0   IO-APIC  22-fasteoi   ohci_hcd:usb1
NMI:          0          0          0          0          0          0          0          0   Non-maskable interrupts
LOC:      76020      98046      95846     106403      75982      86155      79025      84021   Local timer interrupts
SPU:          0          0          0          0          0          0          0          0   Spurious interrupts
PMI:          0          0          0          0          0          0          0          0   Performance monitoring interrupts
IWI:          0          0          0        919          0          0          2         27   IRQ work interrupts
RTR:          0          0          0          0          0          0          0          0   APIC ICR read retries
RES:        750       1415       1379       1173       1034        989       1025       1102   Rescheduling interrupts
CAL:      93453      92933      76474      76778      75659      75309      65452      66436   Function call interrupts
TLB:        403        679        520        451        421        465        457        429   TLB shootdowns
TRM:          0          0          0          0          0          0          0          0   Thermal event interrupts
THR:          0          0          0          0          0          0          0          0   Threshold APIC interrupts
DFR:          0          0          0          0          0          0          0          0   Deferred Error APIC interrupts
MCE:          0          0          0          0          0          0          0          0   Machine check exceptions
MCP:          4          4          4          4          4          4          4          4   Machine check polls
ERR:          0
MIS:         22
PIN:          0          0          0          0          0          0          0          0   Posted-interrupt notification event
NPI:          0          0          0          0          0          0          0          0   Nested posted-interrupt event
PIW:          0          0          0          0          0          0          0          0   Posted-interrupt wakeup event
```

```
===== /proc snapshot =====

 CPU |      IRQ NUMBER |   OBSERVED TIMES |     AVERAGE TIME |      TOTAL TIME |                 DEVICE NAME
-------------------------------------------------------------------------------------------------------------
   3 |              15 |               42 |           164397 |         6904697 |                    ata_piix
   3 |              20 |               75 |            52015 |         3901164 |                   vboxguest
   3 |              21 |               68 |           104024 |         7073691 |          ahci[0000:00:0d.0]
   3 |              21 |               68 |            20129 |         1368836 |                snd_intel8x0
   7 |               1 |              153 |           103497 |        15835074 |                        i8042
   7 |              19 |               62 |            45657 |         2830734 |              ehci_hcd:usb2
   7 |              19 |               62 |            74683 |         4630380 |                    enp0s3
==========================
```

The results show that multiple interrupt sources increased after generating network traffic using ping. While an increase can be observed in interrupts associated with the network interface (enp0s3), additional increases are present in other interrupt sources, including system-level interrupts such as local timers and scheduling-related events.

Using ping, the event tracer for the interrupts was also compared, where I first runned custom tracer trace_pipe, and then compared the output with the raw tracepoints trace_pipe:

Using custom tracer:

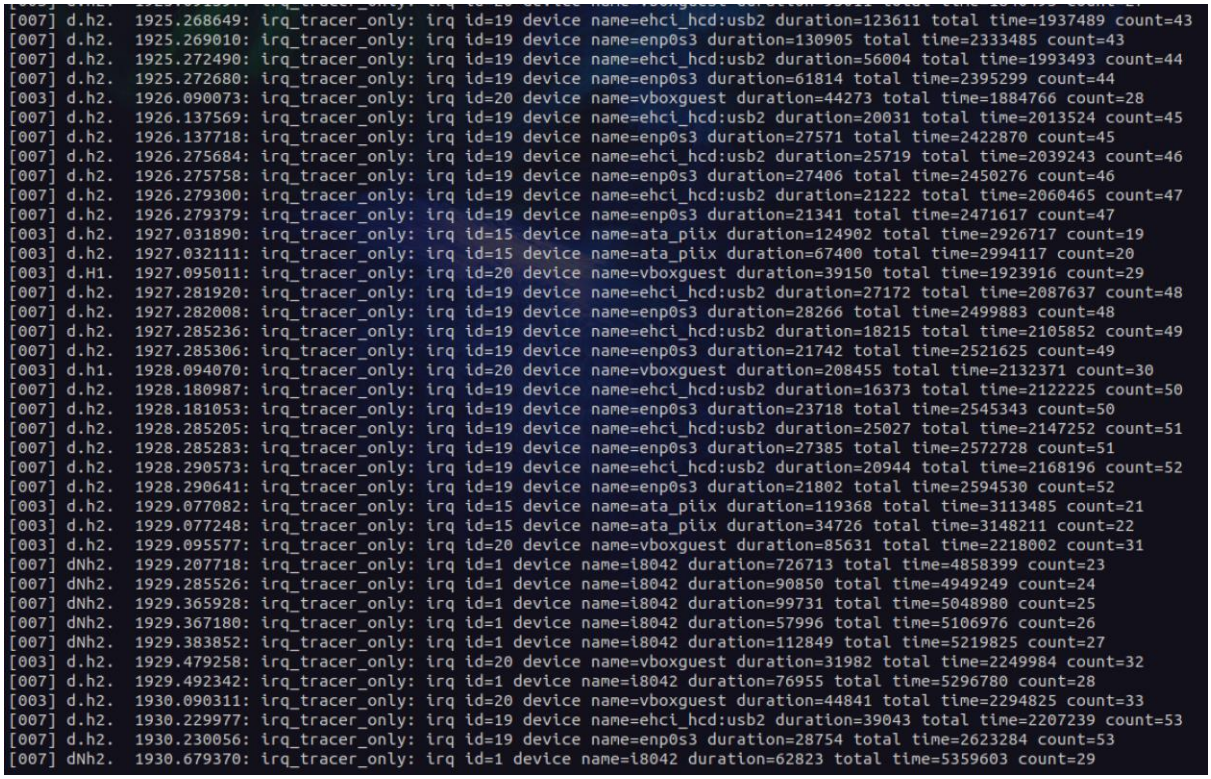
```
[007] d.h2.  1909.205967: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=17533 total time=66821 count=3
[007] d.h2.  1909.206035: irq_tracer_only: irq id=19 device name=enp0s3 duration=23173 total time=76757 count=3
[007] dNh2.  1909.257496: irq_tracer_only: irq id=1 device name=i8042 duration=41987 total time=1863605 count=14
[003] d.h2.  1909.450765: irq_tracer_only: irq id=20 device name=vboxguest duration=57965 total time=218716 count=4
[003] d.h2.  1909.450939: irq_tracer_only: irq id=20 device name=vboxguest duration=39044 total time=257760 count=5
[007] dNh2.  1909.752868: irq_tracer_only: irq id=1 device name=i8042 duration=238221 total time=2101826 count=15
[007] dNH2.  1909.752985: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=16782 total time=83603 count=4
[007] dNH2.  1909.753010: irq_tracer_only: irq id=19 device name=enp0s3 duration=24016 total time=100773 count=4
[007] dNh2.  1909.834629: irq_tracer_only: irq id=1 device name=i8042 duration=624421 total time=2726247 count=16
[007] dNh1.  1909.908573: irq_tracer_only: irq id=1 device name=i8042 duration=292557 total time=3018804 count=17
[007] dNh1.  1909.909548: irq_tracer_only: irq id=1 device name=i8042 duration=48429 total time=3067233 count=18
[003] d.H1.  1910.090657: irq_tracer_only: irq id=20 device name=vboxguest duration=36372 total time=294132 count=6
[007] d.h2.  1910.205019: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=22476 total time=106079 count=5
[007] d.h2.  1910.205094: irq_tracer_only: irq id=19 device name=enp0s3 duration=28718 total time=129491 count=5
[007] d.h1.  1910.210951: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=16425 total time=122504 count=6
[007] d.h1.  1910.211028: irq_tracer_only: irq id=19 device name=enp0s3 duration=34148 total time=163639 count=6
[003] d.h2.  1910.644527: irq_tracer_only: irq id=15 device name=ata_piix duration=117641 total time=284822 count=3
[003] d.h2.  1910.644707: irq_tracer_only: irq id=15 device name=ata_piix duration=52519 total time=337341 count=4
[003] dNh2.  1911.093314: irq_tracer_only: irq id=20 device name=vboxguest duration=311297 total time=605429 count=7
[007] d.h2.  1911.211863: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=29025 total time=151529 count=7
[007] d.h2.  1911.211942: irq_tracer_only: irq id=19 device name=enp0s3 duration=30369 total time=194008 count=7
[007] d.h2.  1911.216481: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=19179 total time=170708 count=8
[007] d.h2.  1911.216550: irq_tracer_only: irq id=19 device name=enp0s3 duration=22293 total time=216301 count=8
[007] d.h2.  1911.797928: irq_tracer_only: irq id=19 device name=ehci_hcd:usb2 duration=39520 total time=210228 count=9
[007] d.h2.  1911.798051: irq_tracer_only: irq id=19 device name=enp0s3 duration=42464 total time=258765 count=9
[003] d.h2.  1912.091369: irq_tracer_only: irq id=20 device name=vboxguest duration=32668 total time=638097 count=8
[003] d.h2.  1912.184143: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=402831 total time=4895400 count=9
[003] d.h2.  1912.184224: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=44408 total time=193053 count=9
[003] d.h2.  1912.184671: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=92184 total time=4987584 count=10
[003] d.h2.  1912.184687: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=15882 total time=208935 count=10
[003] d.h2.  1912.184880: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=112470 total time=5100054 count=11
[003] d.h2.  1912.184928: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=16714 total time=225649 count=11
[003] d.h2.  1912.185115: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=91225 total time=5191279 count=12
[003] d.h2.  1912.185131: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=15037 total time=240686 count=12
[003] d.h2.  1912.192705: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=104841 total time=5296120 count=13
[003] d.h2.  1912.192771: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=18280 total time=258966 count=13
[003] d.h2.  1912.193033: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=156688 total time=5452808 count=14
[003] d.h2.  1912.193080: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=15771 total time=274737 count=14
[003] d.h2.  1912.193285: irq_tracer_only: irq id=21 device name=ahci[0000:00:0d.0] duration=107414 total time=5560222 count=15
[003] d.h2.  1912.193333: irq_tracer_only: irq id=21 device name=snd_intel8x0 duration=15569 total time=290306 count=15
```

Using raw tracepoints and trace_pipe:

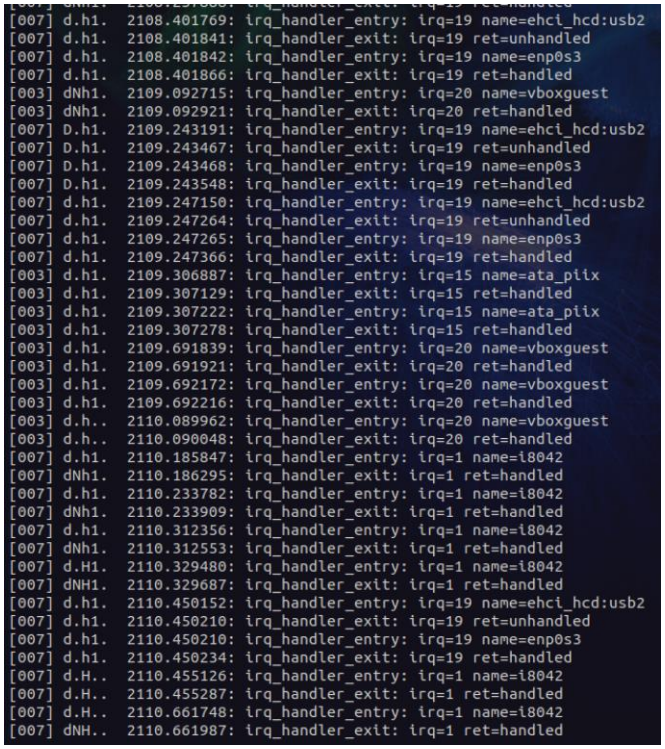
```
[003] d.h1.  2106.112015: irq_handler_entry: irq=21 name=snd_intel8x0
[003] d.h1.  2106.112032: irq_handler_exit: irq=21 ret=unhandled
[003] d.h1.  2106.112762: irq_handler_entry: irq=21 name=ahci[0000:00:0d.0]
[003] d.h1.  2106.113846: irq_handler_exit: irq=21 ret=handled
[003] d.h1.  2106.113846: irq_handler_entry: irq=21 name=snd_intel8x0
[003] d.h1.  2106.113862: irq_handler_exit: irq=21 ret=unhandled
[003] d.h1.  2106.114426: irq_handler_entry: irq=21 name=ahci[0000:00:0d.0]
[003] d.h1.  2106.114566: irq_handler_exit: irq=21 ret=handled
[003] d.h1.  2106.114566: irq_handler_entry: irq=21 name=snd_intel8x0
[003] d.h1.  2106.114584: irq_handler_exit: irq=21 ret=unhandled
[003] dNh1.  2106.114699: irq_handler_entry: irq=21 name=ahci[0000:00:0d.0]
[003] dNh1.  2106.114792: irq_handler_exit: irq=21 ret=handled
[003] dNh1.  2106.114792: irq_handler_entry: irq=21 name=snd_intel8x0
[003] dNh1.  2106.114807: irq_handler_exit: irq=21 ret=unhandled
[003] d.h1.  2106.115138: irq_handler_entry: irq=21 name=ahci[0000:00:0d.0]
[003] d.h1.  2106.115272: irq_handler_exit: irq=21 ret=handled
[003] d.h1.  2106.115272: irq_handler_entry: irq=21 name=snd_intel8x0
[003] d.h1.  2106.115289: irq_handler_exit: irq=21 ret=unhandled
[007] d.h1.  2106.228530: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.228885: irq_handler_exit: irq=19 ret=unhandled
[007] d.h1.  2106.228888: irq_handler_entry: irq=19 name=enp0s3
[007] d.h1.  2106.229016: irq_handler_exit: irq=19 ret=handled
[007] d.h1.  2106.234386: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.234594: irq_handler_exit: irq=19 ret=unhandled
[007] d.h1.  2106.234595: irq_handler_entry: irq=19 name=enp0s3
[007] d.h1.  2106.234668: irq_handler_exit: irq=19 ret=handled
[007] d.h1.  2106.354962: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.355253: irq_handler_exit: irq=19 ret=unhandled
[007] d.h1.  2106.355255: irq_handler_entry: irq=19 name=enp0s3
[007] d.h1.  2106.355431: irq_handler_exit: irq=19 ret=handled
[007] d.h1.  2106.901295: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.901466: irq_handler_exit: irq=19 ret=unhandled
[007] d.h1.  2106.901467: irq_handler_entry: irq=19 name=enp0s3
[007] d.h1.  2106.901533: irq_handler_exit: irq=19 ret=handled
[007] d.h1.  2106.912913: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.912981: irq_handler_exit: irq=19 ret=unhandled
[007] d.h1.  2106.912981: irq_handler_entry: irq=19 name=enp0s3
[007] d.h1.  2106.913003: irq_handler_exit: irq=19 ret=handled
[007] d.h1.  2106.914474: irq_handler_entry: irq=19 name=ehci_hcd:usb2
[007] d.h1.  2106.914533: irq_handler_exit: irq=19 ret=unhandled
```

Here, the results show that about halfway through tracing the ping command, the interrupts being triggered are similar in both outputs. The same applies towards the end of the ping command and when stopping the trace_pipe output.

Custom tracer:



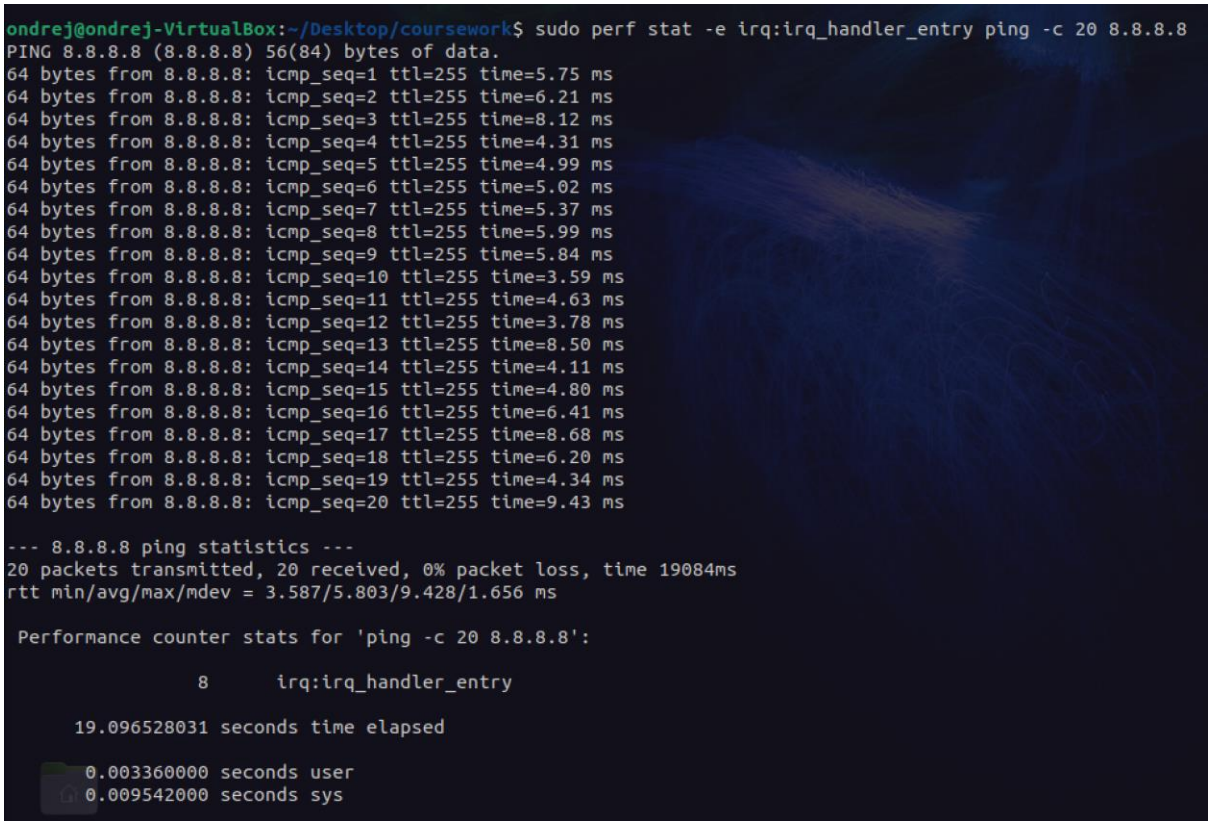
Raw interrupt tracepoints printing via trace_pipe:



Ftrace does not provide the same structured timing output, but it was useful for verifying that the measured durations follow expected patterns and stay within reasonable limits. The collected data is generally consistent across the different sources, so it can be assumed that the tracer captures the required interrupt data correctly.

However, it is important to note that both, ftrace and the custom tracer rely on the same underlying subsystem (trace_pipe and raw tracepoints). Because of this, they cannot be run at the same time for the most accurate comparison. Also, small differences in results may occur due to scheduling and other concurrent system activity.

Additionally, perf was used as a supporting tool to observe overall interrupt activity and system behaviour. Although perf provides statistical rather than exact measurements, it helped confirm that the frequency of interrupts and general system activity matched the observations from the tracer:



As expected, the observed behaviour is similar with both, ftrace and the custom tracer.

Overall, the IRQ tracer demonstrates correct capture of interrupt events, accurate counting across CPUs, and assumably reliable measurement of execution time, including proper handling of nested interrupts.

6.3 Software interrupt tracer validation

The software interrupt tracer was validated by comparing both aggregation mode and event mode outputs with kernel-level sources. Since software interrupt activity is handled internally

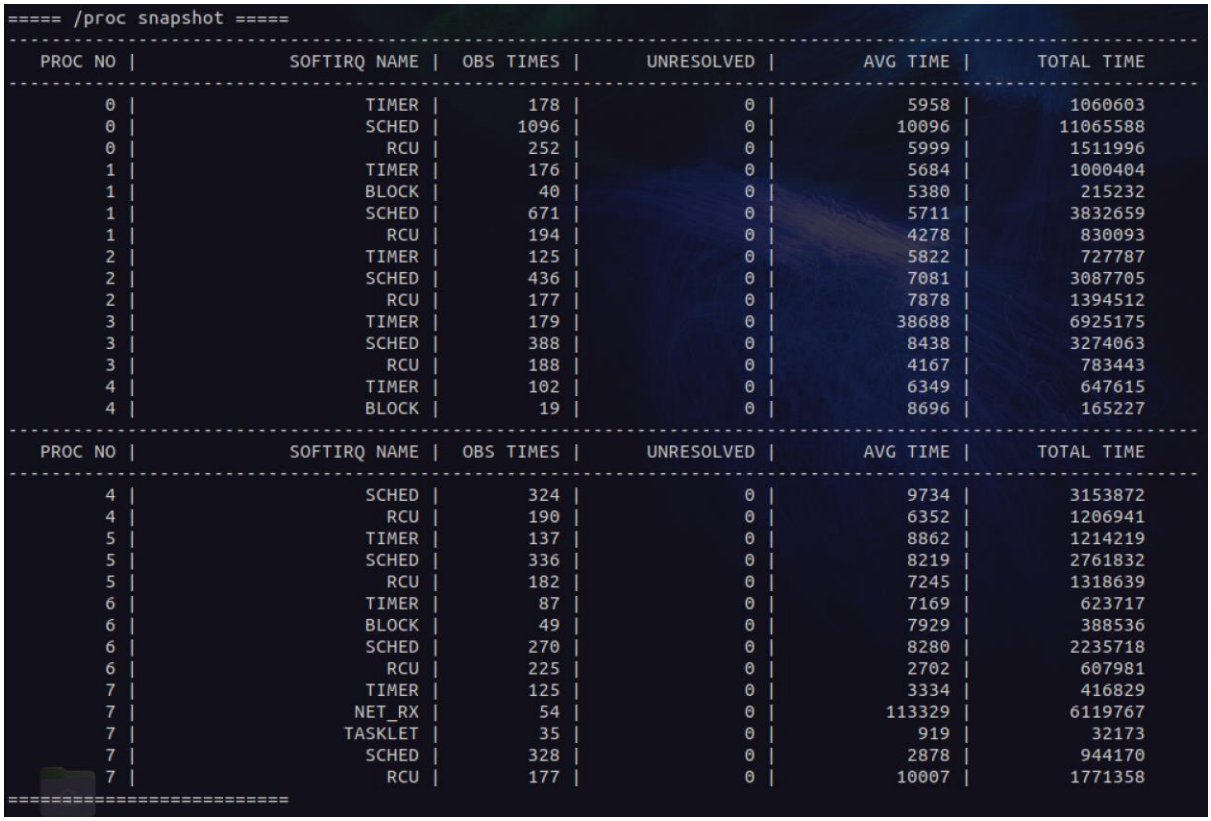
by the kernel and can run across multiple CPUs at the same time, multiple sources were used for validation. The same test scenario was used by running a ping command to Google twenty times. The results were obtained by comparing system activity before and after the ping.

/proc/softirq results before and after ping:

```
ondrej@ondrej-VirtualBox:~$ cat /proc/softirqs
                    CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
          HI:          0          0          1          0          2          0          0          0
       TIMER:      21309      17774      15752      31868      13177      15866      11076      13594
      NET_TX:          3          0          0          0          0          0          3          1
      NET_RX:         14          1         22         58         89         28        144       3384
       BLOCK:       6229       9758      11945       9293       9840      12891       7776       8866
    IRQ_POLL:          0          0          0          0          0          0          0          0
     TASKLET:         24          0          3         21         35          0          0       3095
       SCHED:     110198      52588      36540      49871      28670      31224      25700      34784
     HRTIMER:          0          0          0          0          0          0          0          0
         RCU:      25161      27194      27255      27345      23416      25736      23143      23053
ondrej@ondrej-VirtualBox:~$ cat /proc/softirqs
                    CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
          HI:          0          0          1          0          2          0          0          0
       TIMER:      21500      17967      15891      32063      13271      16031      11174      13708
      NET_TX:          3          0          0          0          0          0          3          1
      NET_RX:         14          1         22         58         89         28        144       3439
       BLOCK:       6229       9798      11946       9293       9858      12891       7826       8866
    IRQ_POLL:          0          0          0          0          0          0          0          0
     TASKLET:         24          0          3         21         35          0          0       3168
       SCHED:     111276      53281      36993      50263      28965      31558      26016      35117
     HRTIMER:          0          0          0          0          0          0          0          0
         RCU:      25501      27394      27459      27553      23610      25939      23410      23244
ondrej@ondrej-VirtualBox:~$
```

My tracer before and after ping:

```
===== /proc snapshot =====
-------------------------------------------------------------------------------------------------------
  PROC NO |          SOFTIRQ NAME |  OBS TIMES |      UNRESOLVED |        AVG TIME |       TOTAL TIME
-------------------------------------------------------------------------------------------------------
        0 |                 TIMER |         14 |               0 |            6205 |           86870
        0 |                 SCHED |         56 |               0 |           16268 |          911011
        0 |                   RCU |         22 |               0 |            3353 |           73770
        1 |                 TIMER |         28 |               0 |            4663 |          130573
        1 |                 SCHED |         28 |               0 |            5896 |          165092
        1 |                   RCU |         21 |               0 |            2986 |           62721
        2 |                 TIMER |          9 |               0 |            7654 |           68891
        2 |                 SCHED |         26 |               0 |            5036 |          130938
        2 |                   RCU |         28 |               0 |            5107 |          143008
        3 |                 TIMER |         17 |               0 |           27629 |          469708
        3 |                 SCHED |         24 |               0 |           16325 |          391807
        3 |                   RCU |         19 |               0 |           10295 |          195623
        4 |                 TIMER |          4 |               0 |            5096 |           20385
        4 |                 SCHED |         17 |               0 |           73206 |         1244518
        4 |                   RCU |         13 |               0 |            2292 |           29803
-------------------------------------------------------------------------------------------------------
  PROC NO |          SOFTIRQ NAME |  OBS TIMES |      UNRESOLVED |        AVG TIME |       TOTAL TIME
-------------------------------------------------------------------------------------------------------
        5 |                 TIMER |          9 |               0 |           11329 |          101969
        5 |                 SCHED |         17 |               0 |            6046 |          102791
        5 |                   RCU |         17 |               0 |            7136 |          121317
        6 |                 TIMER |          7 |               0 |            8997 |           62984
        6 |                 BLOCK |          1 |               0 |            6392 |            6392
        6 |                 SCHED |         21 |               0 |            5104 |          107186
        6 |                   RCU |         92 |               0 |            3258 |          299762
        7 |                 TIMER |          3 |               0 |            2710 |            8130
        7 |                NET_RX |          1 |               0 |           30412 |           30412
        7 |               TASKLET |          2 |               0 |             988 |            1976
        7 |                 SCHED |         19 |               0 |            3617 |           68728
        7 |                   RCU |         22 |               0 |           65656 |         1444443
==========================
```



```
===== /proc snapshot =====
-------------------------------------------------------------------------------------------------------
  PROC NO |            SOFTIRQ NAME |  OBS TIMES |      UNRESOLVED |        AVG TIME |      TOTAL TIME
-------------------------------------------------------------------------------------------------------
        0 |                   TIMER |        178 |               0 |            5958 |          1060603
        0 |                   SCHED |       1096 |               0 |           10096 |         11065588
        0 |                     RCU |        252 |               0 |            5999 |          1511996
        1 |                   TIMER |        176 |               0 |            5684 |          1000404
        1 |                   BLOCK |         40 |               0 |            5380 |           215232
        1 |                   SCHED |        671 |               0 |            5711 |          3832659
        1 |                     RCU |        194 |               0 |            4278 |           830093
        2 |                   TIMER |        125 |               0 |            5822 |           727787
        2 |                   SCHED |        436 |               0 |            7081 |          3087705
        2 |                     RCU |        177 |               0 |            7878 |          1394512
        3 |                   TIMER |        179 |               0 |           38688 |          6925175
        3 |                   SCHED |        388 |               0 |            8438 |          3274063
        3 |                     RCU |        188 |               0 |            4167 |           783443
        4 |                   TIMER |        102 |               0 |            6349 |           647615
        4 |                   BLOCK |         19 |               0 |            8696 |           165227
-------------------------------------------------------------------------------------------------------
  PROC NO |            SOFTIRQ NAME |  OBS TIMES |      UNRESOLVED |        AVG TIME |      TOTAL TIME
-------------------------------------------------------------------------------------------------------
        4 |                   SCHED |        324 |               0 |            9734 |          3153872
        4 |                     RCU |        190 |               0 |            6352 |          1206941
        5 |                   TIMER |        137 |               0 |            8862 |          1214219
        5 |                   SCHED |        336 |               0 |            8219 |          2761832
        5 |                     RCU |        182 |               0 |            7245 |          1318639
        6 |                   TIMER |         87 |               0 |            7169 |           623717
        6 |                   BLOCK |         49 |               0 |            7929 |           388536
        6 |                   SCHED |        270 |               0 |            8280 |          2235718
        6 |                     RCU |        225 |               0 |            2702 |           607981
        7 |                   TIMER |        125 |               0 |            3334 |           416829
        7 |                  NET_RX |         54 |               0 |          113329 |          6119767
        7 |                 TASKLET |         35 |               0 |             919 |            32173
        7 |                   SCHED |        328 |               0 |            2878 |           944170
        7 |                     RCU |        177 |               0 |           10007 |          1771358
==========================
```

In this case, the evaluation focused on the NET_RX softirq, which is related to receiving network packets into the network card. During the workload, an increase (however not significant) in NET_RX activity was observed in both /proc/softirqs and the tracer output. Most of this activity was seen on CPU 7, which is expected, as network interrupts and deferred work are often handled on a single CPU in virtualized environments. A comparison shows that the tracer recorded 53 NET_RX events, while /proc/softirqs showed an increase of 55 events for the same category. This close match confirms that the tracer is correctly capturing network-related softirq activity.

Small differences between the values are expected and can be explained by timing differences, other system activity running at the same time, and the asynchronous nature of software interrupts execution across CPUs. There were also increases in other software interrupt types such as SCHED, TIMER, and RCU, which reflects normal system behaviour during the workload. Overall, the results show that the software interrupt tracer correctly captures their activity, reflects real system behaviour, and provides accurate aggregation across CPUs.

The softirq tracer was also tested in event mode by comparing its output with raw softirq tracepoints from trace_pipe. Because softirqs happen frequently and can run at the same time on different CPUs, the output has many interleaved events, most of which are related to scheduling (SCHED). Because of this, it is not practical to compare individual events one by one. Instead, the validation focused on overall behaviour. During the ping workload, both outputs showed a clear increase in software interrupt activity, confirming that the tracer correctly captures runtime events. The dominance of SCHED softirqs, along with the

presence of others such as NET_RX, matches expected kernel behaviour under load. Even though the output is noisy, the tracer produces stable and continuous events, which shows that it is correctly attached to the kernel tracepoints.

Overall, the results confirm that the event mode reflects software interrupt activity correctly, even under high-frequency and concurrent execution.

6.4 Instruction pointer resolution validation

To further validate the instruction pointer tracer, selected results were compared with reverse-engineered output using Ghidra (National Security Agency, 2024). This was especially useful checking the image resolution part, since standard tracing tools do not provide equivalent information about user-space call-site origin.

For frequently occurring instruction pointers, the resolved image name and computed offset were examined in Ghidra. The offsets reported by the tracer corresponded to valid executable regions (files loaded into the executable through the runtime) within the identified binaries or shared libraries, confirming that the resolution mechanism produced meaningful results.

This comparison does not provide exact one-to-one proof of every call-site, but it supports the correctness of the image resolution logic and shows that the tracer can link system call activity to realistic user-space code locations.

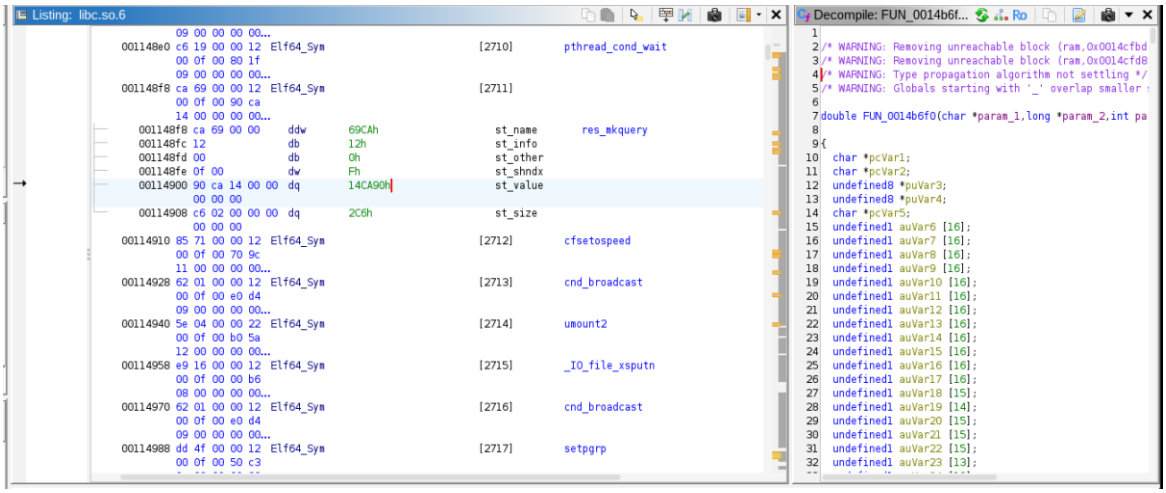
To compare the outputs, I have called simple program that prints simple output 1000 times:

```
ondrej@ondrej-VirtualBox:~/Desktop/coursework$ sudo ./tracer.out -m ip_summary -p 4729 -s 1
Loading module: sudo insmod aggregated_mode/ip_summary/ip_summary_mod.ko target_pid=4729 target_syscall=1
Tracing... (p = print /proc (agg mode only), q = quit)

===== /proc snapshot =====
No entries.
==========================

===== /proc snapshot =====
Top 1 hottest syscall call-sites
-------------------------------------------------------------------------------------------------------------------------------------------
   rank |    count |            pid |    syscall |            syscall name |            ip |          offset |          comm |          image
-------------------------------------------------------------------------------------------------------------------------------------------
      1 |     1000 |           4729 |          1 |                   write | 0x 7e232db14907 | 0x       114907 |         test2 |       libc.so.6
==========================

Stopping tracer...
ondrej@ondrej-VirtualBox:~/Desktop/coursework$
```



During evaluation, system call instruction pointers were consistently resolved to shared libraries, most commonly libc.so.6 for write instruction, rather than the user-defined application binary. This was expected, as standard system calls in user-space applications are typically invoked through library wrapper functions.

For example, functions such as write() are implemented in the C standard library, which internally executes the system call instruction. Because of this, the tracer captures the actual execution point of the system call inside the library, rather than the original call site in the application code.

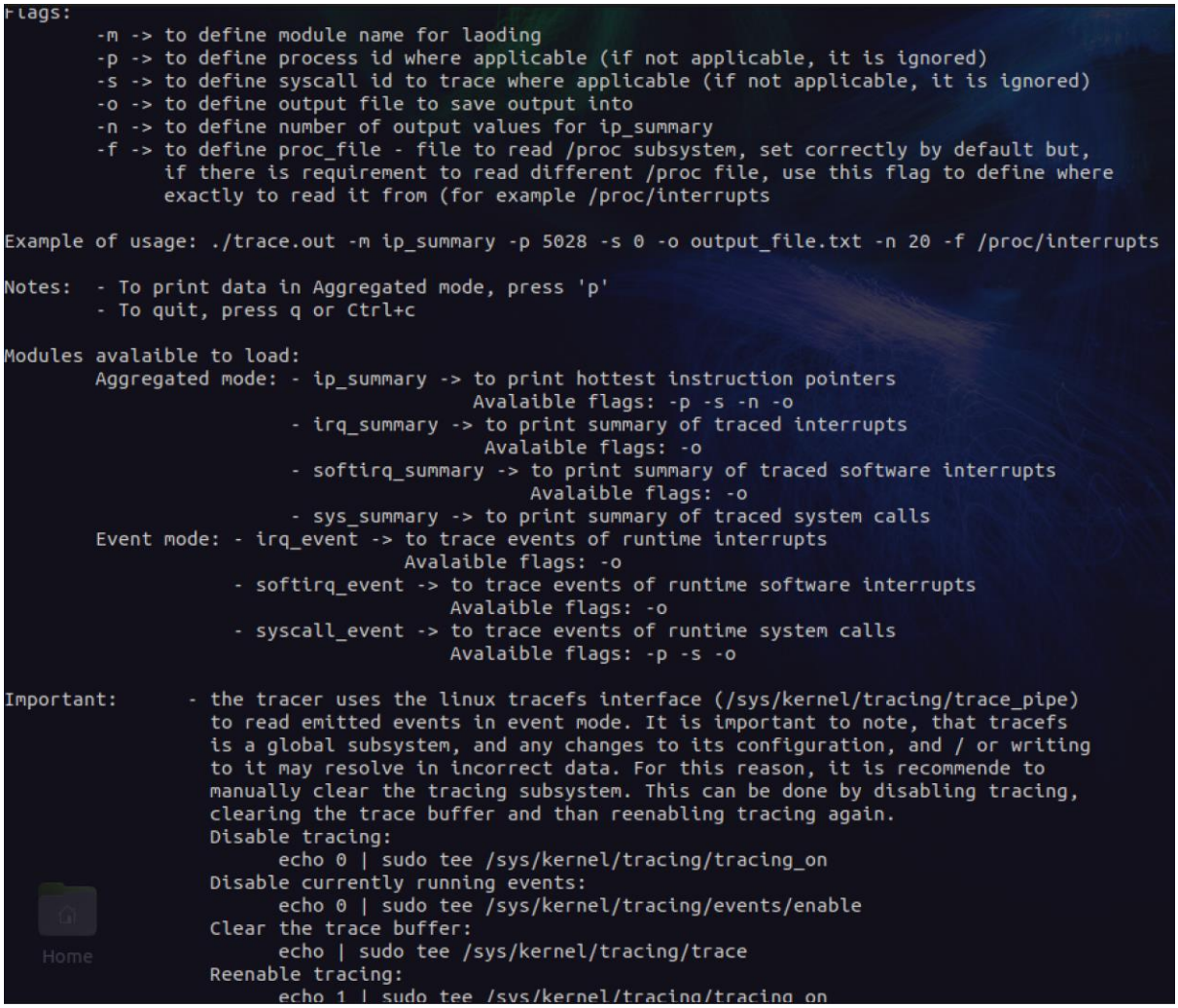
This shows an important difference between the logical origin of a system call (where it is invoked in the application) and the actual execution origin (where the system call instruction is executed). The tracer correctly captures the latter, reflecting the true low-level behaviour of the system.

However, it is also important to note that the instruction pointer resolution is not fully precise, as it does not show the exact implementation details inside the library itself. It only resolves the instruction pointer to the library and a general offset, rather than a specific function or line within the library code.

Although this limits direct mapping of system calls back to specific lines of user code, it confirms that the tracer accurately identifies the execution context in which system calls occur.

6.5 Usability and design Evaluation

In addition to correctness and performance, development was focused into usability and overall design. The system was designed with a focus on simplicity and modularity, allowing individual components to be tested and used independently. From a user's perspective, the tracer is controlled through a simple command-line interface, where users can select modules and specify parameters such as process ID or system call filters. This makes it easy to configure without needing extra tools or complex setup. The ability to dynamically load and unload modules also improves usability, as tracing can be enabled or disabled at runtime.



The modular design of the tracer was helpful during both development and testing. Each module (system calls, interrupts, software interrupts, and instruction pointer analysis) works independently, which made it easier to isolate issues and debug specific parts of the system. This structure also makes it easier to extend in the future without needing major changes to the existing code.

Output was also an important part of the design. Aggregated results, that were exposed through the /proc filesystem, are formatted into readable tables, while event-based data is available through the standard tracing infrastructure. This means the tracer provides useful information without requiring extra processing.

However, there are still some usability limitations. The tracer requires root privileges to load kernel modules and access the tracing system, and it assumes some basic understanding of Linux kernel concepts. However, the overall design offers a good balance between functionality, flexibility, and ease of use.

7. Conclusion, educational contribution, limitations and future

7.1 Conclusion

This project explored the design and implementation of a modular kernel-level tracing system using Linux tracepoints. Through developing and evaluating tracers for system calls, interrupts, software interrupts, and instruction pointers, it became clear how complex and dynamic kernel behaviour is, especially under real workloads and concurrent execution.

One of the main takeaways is a better understanding of how different layers of the operating system interact. In particular, the difference between user-space abstractions and actual kernel execution became clear during the development of each module. Another important aspect of this project was working with concurrency and multi-core execution. Tracing interrupts and software interrupts showed that kernel activity is highly parallel and often non-deterministic. This made validation more challenging and required careful reasoning about per-CPU execution, timing differences, and background system activity. Overall, this provided useful insight into how real systems behave beyond simplified models.

7.2 Educational Contribution

In addition to the results, this project provided valuable practical experience with low-level system programming and kernel instrumentation. Working in kernel space required a deeper understanding of core operating system concepts such as system calls handling, interrupt handling, and deferred execution. One of the most important learning outcomes was understanding concurrent and multi-core behaviour, where execution is not always predictable. This highlighted the need for proper synchronization, correct data structures, and careful design used when working in parallel environments.

The project also reinforced the gap between theoretical knowledge and real system behaviour. Concepts that are usually introduced at a high level were explored in detail through direct interaction with kernel tracing mechanisms. This hands-on experience helped build a clearer understanding of how software interacts with the operating system at a low level.

Overall, the project helped develop practical skills in debugging, performance analysis, and system-level thinking.

7.3 Limitations

Despite achieving the main goals, several limitations were identified. One key limitation is the reliance on the shared tracing infrastructure (tracefs), which makes it difficult to directly compare results with tools like ftrace. Since both use the same underlying mechanisms, it is not always possible to run them at the same time for perfectly accurate comparisons. Small differences in timing and counts are also expected due to scheduling and concurrent system activity.

Another limitation is that the modules were not designed for long-term execution under heavy workloads. While they work reliably in controlled tests, longer runtime or very high event frequency could lead to issues such as counter overflows or instability.

There are also limitations in instruction pointer resolution. While the tracer correctly identifies the related binary or shared library executing specific system call, the reported offsets are based on virtual memory mappings rather than exact file layouts. This makes comparisons with reverse engineering tools less precise.

Exception tracing was not included in this project. Compared to interrupts and software interrupts, exceptions are more complex and involve deeper interaction with architecture-specific kernel behaviour. Due to time limitations, this was considered out of scope.

Finally, the use of hardcoded system call and software interrupt names introduces a dependency on the specific kernel version. Changes between kernel versions may require updates to maintain compatibility.

7.4 Future work

Future improvements could focus on better data structures within the kernel modules. During development, challenges related to concurrency showed how important it is to choose the right structures for safe and efficient data handling. Improving this could increase both performance and reliability, especially under heavy workloads. Other possible improvements include more accurate instruction pointer resolution, better filtering options, and support for more advanced tracing scenarios.

Beyond technical improvements to the tracer itself, this project has also highlighted a strong interest in continuing development in system-level software. Future work may involve exploring lower-level programming in more depth, particularly x86_64 architectures, as well as gaining a deeper understanding of hardware architecture and how it interacts with the operating system. This would help build a more complete understanding of system behaviour from the hardware level up towards the user space.

Overall, the project shows that it is possible to build a flexible and practical tracing system using Linux tracepoints, while also providing useful insight into kernel behaviour and system-level analysis.

Appendix

Appendix A - Kernel modules lifecycles and tracepoint registrations

Kernel modules are loaded using module_init and module_exit functions, using __init and __exit macros. Once module is required to be loaded, registered functions (in this case sys_trace_init) are executed. On the unload of the module, sys_trace_exit is called.

```
module_param(target_pid, int, 0644);
module_param(target_syscall, int, 0644);
module_param(top_n, int, 0644);
module_init(sys_trace_init);
module_exit(sys_trace_exit);
MODULE_LICENSE("GPL");
```

To register required function into the tracepoint:

```
tracepoint_probe_register(tp_sys_enter, sys_trace_enter, NULL);
```

This registeres sys_trace_enter function into the tracepoint, therefore once the tracepoint fires, this function is called (to fetch or update any required data). The same procedure is used for exit tracepoints.

Appendix B - /proc usage in aggregation mode

Implementation uses proc_fs and seq_file (commonly used in kernel modules). /proc entry is created on the module loadup (__init).

```
/* Struct for proc operations */
static const struct proc_ops ip_proc_ops = {
    .proc_open   = bridge_print_ip,
    .proc_read   = seq_read,
    .proc_lseek      = seq_lseek,
    .proc_release    = single_release,
};
```

The proc_ops structure defines how the kernel should handle operations on this file. Here, the important part is ‘bridge_print_ip’, which connects the /proc file to the function responsible for the output.

```
/* Function to bridge print_sys with the function that creates proc file
   !!must be this signature!! */
static int bridge_print_ip(struct inode *inode, struct file *file)
{
    return single_open(file, print_ip, NULL);
}
```

Because the seq_file interface expects a specific signature, small bridge function is required to bridge .proc_open with print_ip function.

Appendix C - Tracepoint definition and registration using TRACE_EVENT

Unlike /proc file subsystem, TRACE_EVENT writes directly into the tracing per-CPU ring-buffer. Tracepoint is defined using TRACE_EVENT macro. This macro generates a complete pipeline, including data structures, registration hook and formatting logic.

```c
/* TRACE_EVENT  -> defines a tracepoint called irq_traer_only
 * TP_PROTO -> function prototype -> basically just a signature on how i call trace_irq_tracer_only form module
 * TP_ARGS  -> arguments passed internally */
TRACE_EVENT(irq_tracer_only,
    TP_PROTO(int irq_id, const char *device_name, u64 exe_time, u64 total_time, u64 observed_times),
    TP_ARGS(irq_id, device_name, exe_time, total_time, observed_times),

    /* Structure stored inside the ringbuffer */
    TP_STRUCT__entry(
        __field(int, irq_id)            // irq number
        __string(device_name, device_name)  // device name
        __field(u64, exe_time)          // execution time of irq
        __field(u64, total_time)        // total time of executing this irqs
        __field(u64, observed_times)        // count of this irqs
    ),

    /* Assign values from arguments into the struct */
    TP_fast_assign(
        __entry->irq_id = irq_id;
        __assign_str(device_name, device_name);
        __entry->exe_time = exe_time;
        __entry->total_time = total_time;
        __entry->observed_times = observed_times;
    ),

    /* Formated output into tracebuffer */
    TP_printk("irq id=%d device name=%s duration=%llu total time=%llu count=%llu",
        __entry->irq_id,
        __get_str(device_name),
        __entry->exe_time,
        __entry->total_time,
        __entry->observed_times
    )
);
```

TP_PROTO - defines the function signature, that will be used when the event is triggered from the kernel module (just write types of all data that are to be exposed).

TP_ARGS - specifies how arguments are passed internally to the tracing infrastructure (repeat names of the variables to pass).

TP_STRUCT_entry - section defines the data structure that will be stored inside the trace ring buffer.

TP_fast_assign - executes, when the tracepoint is triggered. It copies values from the arguments into the internal trace buffer structure.

TP_printk - defines, how that structure will be printed, once ring buffer is called to be printed.

To make this tracepoint usable, the kernel must generate required supporting code. This is achieved by including the trace definition footer.

```c
/* Define trace subsystem name (appears in trace output)*/
#undef TRACE_SYSTEM
#define TRACE_SYSTEM irq_tracer

/* Header guard that is required for tracepoint headers*/
#if !defined(_TRACE_IRQ_TRACER_H) || defined(TRACE_HEADER_MULTI_READ)
#define _TRACE_IRQ_TRACER_H

#include <linux/tracepoint.h>
```

This inclusion triggers macro, so I can call it and write data from the kernel module using trace_irq_tracer_only().

```c
/* expose data into trace buffer to be printed */
trace_irq_tracer_only(
    irq,                        // irq number
    result->device_name,                // device name
    current_time - finished.starting_time,      // time spend executin interrupt
    result->total_time,             // total time spent on all irqs
    result->observed_times              // count
);
```

After all of the registration and data exposures, the tracepoint has to be enabled explicitly through the tracing subsystem: echo 1 > /sys/kernel/tracing/events/irq_tracer/enable

Reference list

Beamonte, R., Giraldeau, F. and Dagenais, M.R. (2012) *High Performance Tracing Tools for Multicore Linux Hard Real-Time Systems*. Montréal: École Polytechnique de Montréal.

Bovet, D.P. and Cesati, M. (2005) *Understanding the Linux Kernel*. 3rd edn. Sebastopol: O'Reilly Media.

Esteves, T., Macedo, R., Oliveira, R. and Paulo, J. (2023) 'Diagnosing applications' I/O behavior through system call observability', in *Proceedings of the 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE, pp. 1–8. doi:10.1109/DSN-W58399.2023.00022.

Gebai, M. and Dagenais, M.R. (2018) 'Survey and analysis of kernel and userspace tracers on Linux: Design, implementation, and overhead', *ACM Computing Surveys*, 51(2), Article 26. doi:10.1145/3158644.

Hausdorf, A., Hinzmann, N. and Zeckzer, D. (2019) 'SyCaT-Vis: Visualization-based support of analyzing system behavior based on system call traces', in *Proceedings of WSCG 2019*. Computer Science Research Notes, pp. 45–54. doi:10.24132/CSRN.2019.2901.1.6.

Linux Manual Pages Project (2024) *perf(1) – Linux manual page*. Available at: https://man7.org/linux/man-pages/man1/perf.1.html (Accessed: 15 February 2026).

Linux Manual Pages Project (2025a) *ptrace(2) – Linux manual page*. Available at: https://man7.org/linux/man-pages/man2/ptrace.2.html (Accessed: 15 February 2026).

Linux Manual Pages Project (2025b) *strace(1) – Linux manual page*. Available at: https://man7.org/linux/man-pages/man1/strace.1.html (Accessed: 15 February 2026).

Love, R. (2010) *Linux Kernel Development*. 3rd edn. Indianapolis: Addison-Wesley.

Mauerer, W. (2008) *Professional Linux Kernel Architecture*. Indianapolis: Wiley.

National Security Agency (2024) *Ghidra Software Reverse Engineering Framework*. Available at: https://ghidra-sre.org/ (Accessed: 9 April 2026).

The Linux Kernel Developers (2024a) *BPF Documentation*. Available at: https://docs.kernel.org/bpf/ (Accessed: 13 February 2026).

The Linux Kernel Developers (2024b) *DebugFS*. Available at: https://docs.kernel.org/filesystems/debugfs.html (Accessed: 13 February 2026).

The Linux Kernel Developers (2024c) *Event Tracing*. Available at: https://docs.kernel.org/trace/events.html (Accessed: 13 February 2026).

The Linux Kernel Developers (2024d) *ftrace - Function Tracer*. Available at: https://docs.kernel.org/trace/ftrace.html (Accessed: 13 February 2026).

The Linux Kernel Developers (2024e) *Kernel Probes (Kprobes)*. Available at: https://docs.kernel.org/trace/kprobes.html (Accessed: 13 February 2026).

The Linux Kernel Developers (2024f) *Lockless Ring Buffer Design*. Available at: https://docs.kernel.org/trace/ring-buffer-design.html (Accessed: 13 February 2026).

The Linux Kernel Developers (2024g) *Using the Linux Kernel Tracepoints*. Available at: https://docs.kernel.org/trace/tracepoints.html (Accessed: 13 February 2026).